SENTINEL-GNSS — Individual Poster

Data Engineering: ROSBAG Extraction, RTKLIB Pipeline, Feature Engineering, Dataset

Ogunlade Joshua (LS2525237) Data Engineer

Beihang University H3I · 2025

My Role

I built the complete data engineering pipeline that all model training rests on:

- ROSBAG inspection and GNSS topic identification
- `extract_gnss.py`: ROSBAG → CSV extraction
- `run_rtklib.py`: automated RTKCONV + RTKPOST
- `extract_features.py`: 37 features per epoch
- `assemble_dataset.py`: master dataset assembly, labelling, sliding-window construction
- On-site RTKLIB quality check during all collection sessions
- Dataset quality assurance: histograms, NaN handling, leakage check

ROSBAG Extraction

Supervisor datasets (vehicle + drone) stored as ROS bags. I identified GNSS topics (`/fix`, `/ublox/fix`, `/gnss/raw`) and wrote the extraction script to produce per-epoch CSVs:

- Timestamp, lat/lon/alt, per-satellite SNR and elevation
- Pseudorange residuals where available
- Verified against expected epoch count per dataset

RTKLIB Pipeline

`run_rtklib.py` automates:

1. RTKCONV → RINEX 3.x obs + nav files
2. SP3/CLK download → IGS precise orbits (CDDIS)
3. RTKPOST → SPP solution `.pos` file

Settings:

- Trimble: GPS + GLONASS, dual-frequency
- u-blox F9P: GPS L1 single-frequency
- Elevation mask 10°, Saastamoinen troposphere

Applied to: 5 Beihang scenarios + 4 UrbanNav HK environments
Visual inspection in RTKPLOT for all runs.

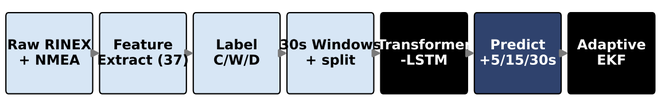

Figure 1: The full data pipeline from raw RINEX/NMEA to labelled feature windows — implemented by Joshua.

Feature Engineering (37 features)

Group	#	Key features
Position quality	4	H/V error, PDOP, HDOP
Signal strength	8	Mean C/N_0, per-SV stats
Satellite geometry	6	SV count, elevation stats
Pseudorange	7	Residuals, Δpseudorange
Temporal	5	10-s rolling mean/std
Atmospheric	4	Tropo/iono delays
Receiver status	3	`receiver_tier`, lock time

Bug fixed: Delta-pseudorange at session boundaries produced ±500 m artifacts. Fixed with 3-point median filter. Affected ≈60 windows/session.

Dataset Assembly & Labelling

3-class labelling criteria:

- **CLEAN**: error < 2 m AND $C/N_0 > 35$ dB-Hz
- **DEGRADED**: error > 5 m OR $C/N_0 < 30$ OR SV < 4
- **WARNING**: all other epochs

Sliding window: $(t-59, \ldots, t) \rightarrow$ predict at $t+5, t+15, t+30$

Session-level split (70/15/15) — prevents temporal leakage: no epoch from a test drive appears in any training window.

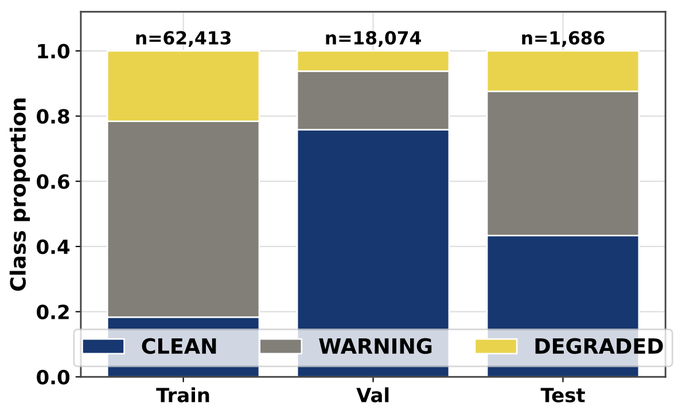

Figure 2: Class distribution per split. DEGRADED: 7.8 % of training windows. Handled by focal loss + class weighting.

Quality Assurance

Issues found and fixed:

1. $C/N_0 = 0$ when SV count > 0: firmware artifact → imputed with session running mean
2. Negative PDOP (RTKLIB extrapolation artifact) → clipped to minimum 1.0
3. Delta-pseudorange boundary spikes → median filter

Leakage verification:

- Session IDs non-overlapping across splits
- Scaler fit only on training data
- `scenario_a_r13` within-site overlap disclosed

After QA: **zero NaN rows** across all 149,662 windows.

Final Dataset Stats

Split	Windows	CLEAN %	DEG %
Train	104,763	58.2	7.8
Validate	22,449	57.1	8.4
Test	1,686	43.4	12.4
Total	**149,662**	4 cities · 9+ receivers	

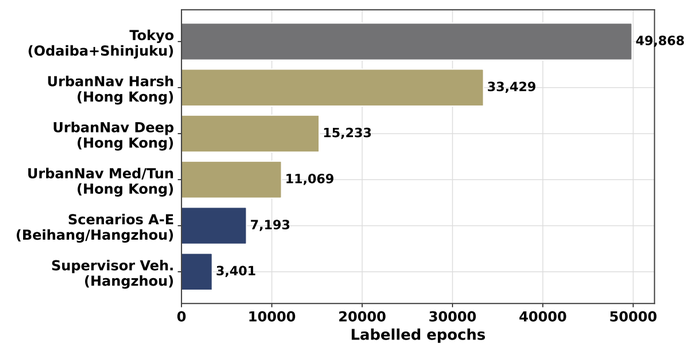

Figure 3: Dataset composition: most diverse labelled GNSS degradation benchmark known to us.

Key Lesson

Session-level split is essential for honest evaluation. A random 70/15/15 split mixes timesteps from the same drive between train and test, artificially inflating metrics by 5–10 % because the model "knows" the transition dynamics of each route. Session-level splitting is the correct methodology for temporal sequence classification in navigation datasets.